Big Data Analysis Classes | 2026Z

Project: CRAN Polish Mirror Demand & GitHub Release Pulse Analysis System

Jakub Grzywaczewski, Mateusz Nizwantowski, Aleks Kapich, Łukasz Grabarski

1. Project Overview & Deliverables

1.1 Description

The CRAN Mirror Demand & Release Pulse Analysis System is an end-to-end Big Data analytics platform engineered to track, analyze, and correlate software downloads from Poland's primary Comprehensive R Archive Network mirror (cran.mi2.ai) with upstream open-source development lifecycles across GitHub. By continuously processing web server access logs and GitHub event feeds, the platform bridges the gap between open-source code commits and real-world package adoption across the Polish data science ecosystem.

The system connects R package artifacts with upstream Git source code using the canonical r-universe crantogit mapping repository, encompassing over 18,000 R packages. This facilitates unified real-time traffic monitoring and multi-year historical trend modeling.

1.2 Purpose, Target Users, and Key Benefits

- **Purpose:** Correlate real-time download events in Poland with upstream GitHub tags and releases, predicting mirror traffic pressure and measuring software adoption velocity.
- **Target Users:** the mirror operator (MI2.ai), R package maintainers, and analysts of the Polish R ecosystem
- **Key Benefits:** the operator sees load and unusual traffic (bots, CI runners) as it happens; maintainers see how quickly a release is picked up in Poland.

1.3 High-Level Deliverables from User's Perspective

- **1. Real-Time Demand Dashboard:** Live Streamlit interface showing top-downloaded R packages across Poland, request velocity, and real-time alerts when new releases trigger download surges.
- **2. Version Adoption & Lag Analytics:** Time-series reports measuring the elapsed lag between a GitHub release tag and peak mirror downloads, including version migration curves.
- **3. Mirror health view**: HTTP errors (404, 5xx), traffic volume per package, share of bot and CI traffic, and active vs dormant packages based on GitHub activity.

2. Core Business Process

2.1 Description

The business workflow operates across three primary actors: the R Package Maintainer / Developer, the CRAN Operations / Mirror Admin, and the Big Data Analytics Engine:

- **1. R Package Maintainer / Developer:** Maintains code on GitHub, tags new versions, and uploads source archives (.tar.gz) to CRAN. Consumes analytics dashboards to evaluate regional adoption rates.
- **2. CRAN Operations / Mirror Admin:** Monitors mirror server health, bandwidth saturation, and package integrity. Adjusts caching rules and plans server capacity based on demand spikes.

- **3. Big Data Analytics Engine:** Streams logs and GitHub webhooks/events, correlates package events via crantogit, maintains rolling window aggregations, and serves reconciled views.

2.2 Formal Business Process Diagram

Figure 1 depicts the complete business process workflow across the three participating swimlanes, formatted with explicit steps, decision gateways, and non-intersecting sequence flows.

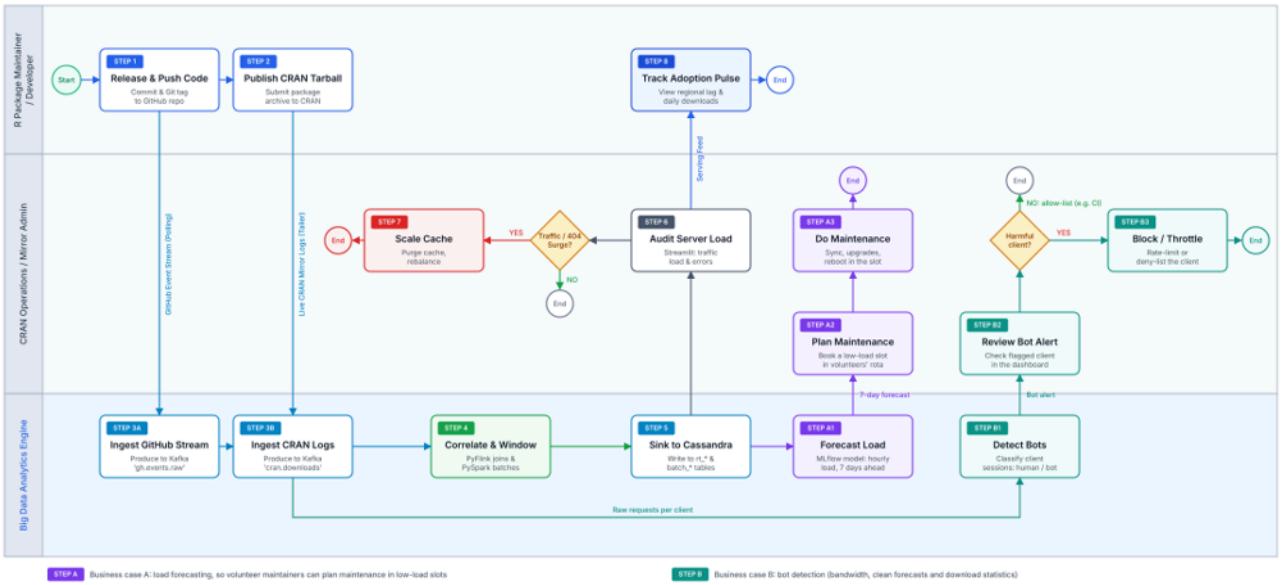


Figure 1: Business Process Swimlane Diagram

3. Preliminary Data Investigation

3.1 Data Sources

- **1. CRAN Mirror Access Logs (cran.mi2.ai):** Live HTTP web logs from cran.mi2.ai capturing GET requests for packages, check results, vignettes, and binaries.
- **2. GitHub Events API - REST/GraphQL Poller:** Public streaming events capturing ReleaseEvent, PushEvent, and Git tags across monitored R repositories.
- **3. crantogit Mapping Registry (r-universe):** Tabular mapping dataset connecting CRAN package names with canonical GitHub repository URLs.

3.2 Data Characteristics & Attributes

Table 1 summarizes data characteristics, followed by Figure 2 which details the regex-based attribute extraction schema applied to each raw mirror log record.

Dataset Name	Data Type	Volume / Velocity	Processing Mode	Format	Key Extracted Attributes

CRAN Mirror Logs (cran.mi2.ai)	Semi-structured (Web Server Logs)	~500-700 MB/month (~2-4 req/sec)	Stream (Live tailing)	Raw W3C/Nginx text log	Client IP, Timestamp, HTTP Verb, URI Path, Package Name, Version, HTTP Status, Size, User-Agent
GitHub Events API (R Watch-list)	Structured REST	~50-100 MB/month (Periodic polling)	Stream	JSON	Repository Name, Event Type (ReleaseEvent, PushEvent), Tag/Commit SHA, Author, Published Timestamp
cran-to-git Mapping (r-universe-org)	Structured tabular metadata	~1 MB total (~18,000 package rows)	Batch (Daily scheduled pull)	CSV	Package Name, GitHub Organization, GitHub Repo URL, Mirror Sync Status, Default Branch

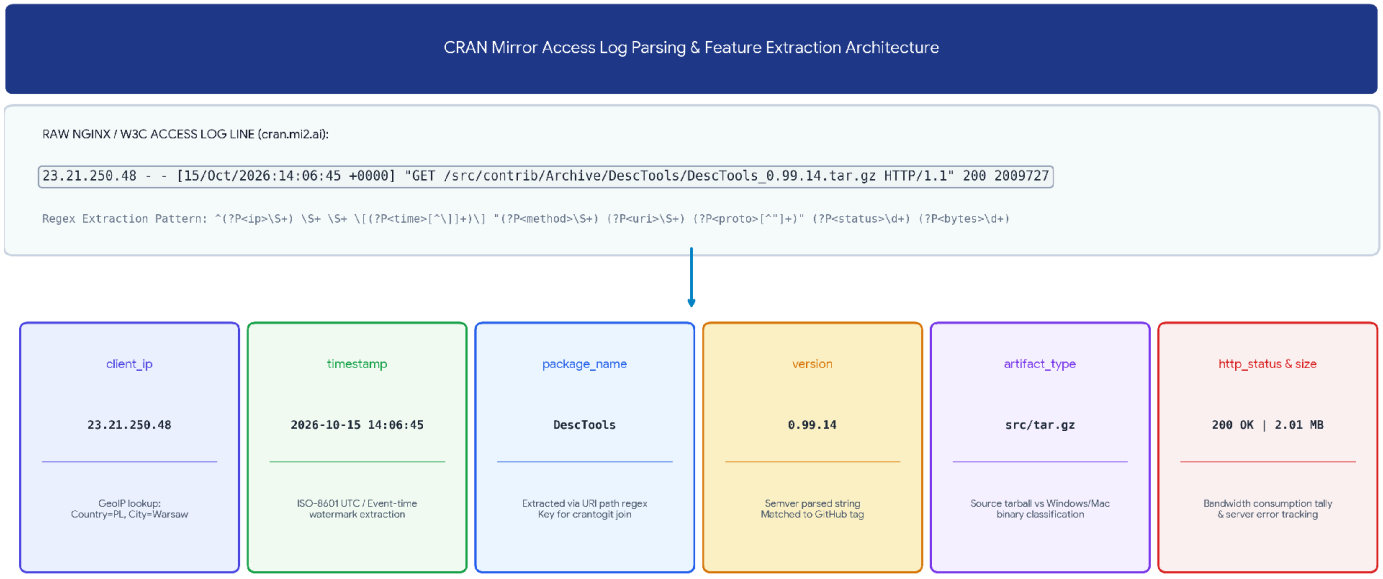


Figure 2: CRAN Access Log Ingestion, Regular Expression Parsing, and Feature Extraction Schema.

3.3 Data Quality & Limitations

- **Bot Filtering & Log Hygiene:** Raw logs include search crawler probes, security scanners, and internal health checks (e.g., 192.168.95.3). Ingestion filters discard non-artifact GET requests to isolate actual package installations.
- **API Quotas:** GitHub imposes an authenticated rate limit of 5,000 req/hr. Polling leverages ETag/If-None-Match headers and prioritizes packages tracked in crantogit.
- **Mapping Coverage:** Around 10-15% of CRAN packages are hosted on platforms outside GitHub (e.g., GitLab, Bioconductor). The pipeline handles unmapped packages gracefully without pipeline failures.

4. Data Processing Assumptions & Architecture

4.1 Stream & Batch Processing Overview (Lambda Architecture)

The project adopts a Lambda Architecture to process both low-latency operational metrics and exhaustive historical trends:

- **Speed Layer (Stream Processing):** Python producers stream access logs into Kafka topic cran.downloads and GitHub events into gh.events.raw. Apache Flink executes stateful windowed aggregations and stream joins, sinking counters with 48h TTL into Cassandra rt_* tables.
- **Batch Layer (Batch Processing):** Kafka streams append immutably to HDFS in Parquet format. Apache Airflow schedules daily batch jobs where PySpark reconciles crantogit metadata, computes multi-year adoption curves, trains MLflow models, and writes to Cassandra batch_* tables.

4.2 System Architecture Diagram (High-Level Data Flow)

Figure 3 illustrates the complete end-to-end Big Data Architecture Diagram, detailing data flows across the Speed Layer (Kafka, PyFlink, Cassandra rt_*), Batch Layer (Airflow, HDFS, PySpark, Cassandra batch_*), ML Layer (MLflow), and Serving Layer (Streamlit).

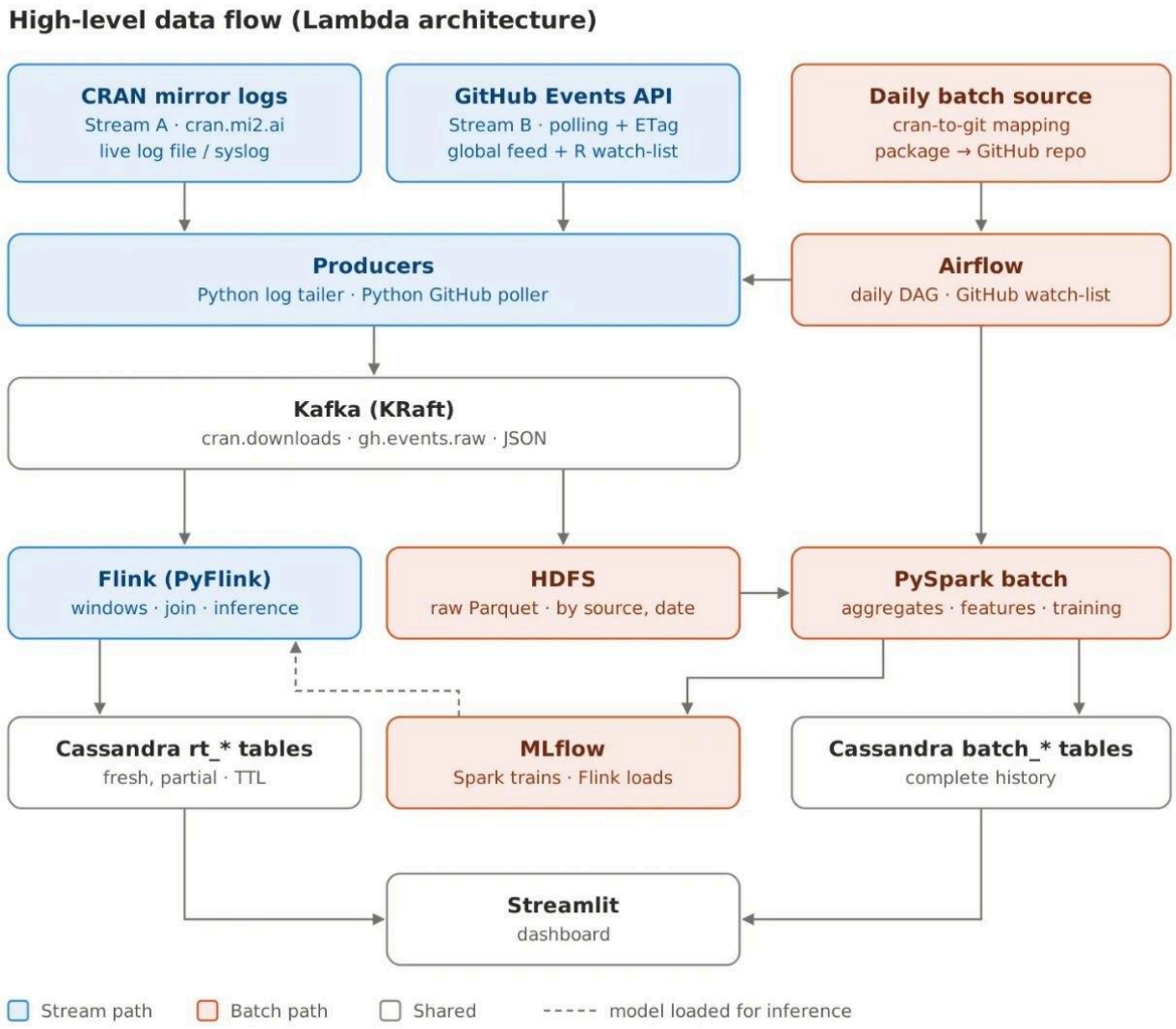


Figure 3: System Architecture DiagramL High-Level Data Flow (Lambda Architecture: Kafka KRaft, PyFlink, HDFS, PySpark, MLflow, Cassandra, Streamlit).

4.3 Key Assumptions & Latency Tolerances

- **Latency Tolerances:** Speed layer streaming latency tolerance is 5 to 10 seconds; batch layer job duration tolerance is up to 1 day.
- **Event Time Semantics:** PyFlink uses event-time watermarking with a 30-second allowed lateness to handle network jitter without data loss.
- **Data Eviction:** Dual-table storage ensures that real-time tables automatically expire transient records, avoiding unbounded disk expansion in Cassandra.

5. Proposed Infrastructure & Big Data Technologies

5.1 Overview of Architecture

The infrastructure leverages open-source distributed frameworks containerized with Docker to enable rapid deployment and local cluster simulation.

5.2 Technologies and Platforms Considered

Architecture Layer	Selected Technology	Role & Justification
Data Ingestion & Event Streaming	Python Producers + Apache Kafka (KRaft mode)	Lightweight daemon tails log files and polls GitHub API with ETag caching. Kafka provides distributed, fault-tolerant publish-subscribe message buffering across partitioned topics without ZooKeeper overhead.
Speed Layer (Stream Processing)	Apache Flink (PyFlink API)	Provides true event-driven stream processing with sub-second latency, robust stateful windowing, dynamic sliding joins between release events and download events, and checkpointing.
Batch Storage Layer	Hadoop Distributed File System (HDFS) / Parquet	Provides scalable, fault-tolerant raw data storage. Parquet columnar encoding maximizes compression efficiency and accelerates vectorised Spark query execution.
Batch Compute & Feature Engineering	Apache Spark (PySpark)	Executes distributed large-scale SQL transformations, rolling historical aggregates, and feature matrix construction over years of log files.
Workflow Orchestration	Apache Airflow	Manages dependencies across daily cron jobs: pulls upstream crantogit metadata, triggers PySpark batch transformations, evaluates data quality, and refreshes Cassandra batch views.
Serving & Storage Layer	Apache Cassandra	Distributed NoSQL wide-column store offering high write-throughput for streaming metrics, tunable consistency, flexible TTL eviction for real-time tables, and low-latency point queries.
ML Tracking & Registry	MLflow	Tracks training experiments, adoption prediction model parameters, version artifacts, and handles model distribution between Spark batch training and Flink streaming inference.
Interactive Visualization	Streamlit	Python-native responsive web application framework providing real-time data streaming widgets, historical trend exploration, and maintainer analytics.

6. Planned Use of GenAI Tools

Generative AI tools are used to research data sources, draft project documentation and diagrams, and generate code. Each AI-assisted task follows the same workflow: the responsible member writes a prompt with the task requirements and source materials, checks the output against primary source & reiterates when a necessity occurs, leading the process towards creating a relevant result. Conversations that produced part of the solution are downloaded for the reporting purposes.

7. Team Members & Task Allocation

The project is executed by a four-member engineering team. Table 3 outlines the division of responsibilities across the architecture:

Task / Responsibility	Jakub Grzywaczewski (PM)	Mateusz Nizwantowski	Aleks Kapich	Łukasz Grabarski
High-level architecture definition & Kafka KRaft deployment	R		S	S
PyFlink pipelines: log parsing, model inference, windowed joins, Cassandra sink	R	S		S
Project documentation leadership & system integration	R	S	S	
CRAN log acquisition daemon & crantogit batch synchronization	S	R		S
Apache Airflow DAG creation & HDFS Parquet partitioning schemes	S	R	S	
PySpark batch transformations & historical aggregation metrics		R	S	S
GitHub Events API poller with ETag rate-limiting		S	R	S
MLflow setup for tracking package adoption velocity models	S		R	S
Apache Cassandra schemas & setup	S	S	R	
Streamlit analytics dashboard (real-time + historical view integration)		S	S	R
Docker Compose container orchestration across all service layers	S		S	R
Pipeline latency benchmarking, stress testing & documentation formatting	S	S		R

8. References

- **1. CRAN Mirror Logs:** Polish CRAN Mirror Access Logs (cran.mi2.ai), Warsaw, Poland. Format: W3C / Nginx Combined Log Format.
- **2. crantogit Dataset:** R-Universe CRAN-to-GitHub Mapping Project. URL: https://github.com/r-universe-org/cran-to-git/blob/master/crantogit.csv
- **3. GitHub Developer Platform:** GitHub REST & Events Streaming API Documentation. URL: https://docs.github.com/en/rest/activity/events
- **4. Big Data Frameworks:** Big Data Frameworks: Apache Kafka (KRaft), Apache Flink, Apache Spark, Apache Airflow, Apache Cassandra, and MLflow official documentation.